

Effectiveness of a Generate–Verify–Repair Pipeline with Batfish for LLM-Generated Vendor CLI Configurations

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We report a preregistered paired experiment (cnsms2026-001-gvr-batfish-prereg-v1) that evaluated whether a generate–verify–repair (GVR) pipeline—shared initial LLM candidate (gpt-5-mini) + a deterministic Batfish-style validator + at most one automated LLM repair—reduces deterministic-verifier detected semantic/safety failures on intent→vendor-CLI tasks. Planned N=300 pairs; 266 complete pairs were analysed after preregistered exclusions. Baseline pass-rate = 260/266 (97.744%); guarded (final GVR) pass-rate = 265/266 (99.624%). Paired difference = 0.01879699248; exact McNemar two-sided p = 0.0625. Paired bootstrap (B=10,000, seed=1729) 95% percentile CI = [0.0037593984962406013, 0.03759398496240601]. Repair was invoked for 6 tasks; median repair iterations per task = 0. Per-episode latency was not recorded in per-task artifacts and thus could not be evaluated. Under shared_initial_candidate semantics the observed absolute improvement is small and did not meet the preregistered $\alpha=0.05$ decision rule. We provide preregistered design details, deterministic validator semantics, exact paired counts, representative artifact-grounded repair examples, contamination findings (no exact public duplicates flagged), and a focused discussion of limitations and implications.

I. INTRODUCTION

Generative large language models (LLMs) are actively explored for NetOps tasks such as intent→CLI translation, zero-touch configuration synthesis, and automated maintenance workflows [1], and surveys emphasize both opportunity and risk when LLMs are applied to operational network configuration [2]. Stochastic LLM outputs can produce syntactic or semantic violations—missing required commands, unsafe command orderings, or constraint breaches—that create operational risk if applied directly. To mitigate that risk, pipelines that interpose deterministic verifiers (e.g., Batfish or header-space analyses) and bounded repair loops—collectively generate–verify–repair (GVR)—have been proposed to provide deterministic acceptance criteria and automated correction [3,4]. Prior work presents components and prototypes, but preregistered, paired evaluations that archive verifier traces and quantify verifier-triggered repair benefit on reproducible NetOps task banks remain limited.

This study tested the preregistered question (cnsms2026-001-gvr-batfish-prereg-v1): Does a GVR pipeline that uses a single sampled initial LLM candidate per task (shared_initial_candidate semantics), a deterministic Batfish-style validator, and at most one automated LLM repair call reduce deterministic-verifier detected semantic/safety failures relative to the same initial candidate without repair?

Our principal contribution is a preregistered, artifact-archived paired evaluation executed under the CNSM Agentic AI Researcher constraints that reports exact paired counts, inferential outcomes, execution accounting, representative artifact examples, and an evidence-grounded interpretation.

II. RELATED WORK

LLM applications in NetOps include intent→CLI translation systems and focused benchmarks studying automated configuration synthesis and correctness [1]. Surveys synthesize common failure modes (including hallucination and constraint violations) and advocate grounding and verification components for automation [2]. Generate–verify–repair architectures have been proposed and explored in code generation and regulated domains to achieve deterministic validation via external oracles and bounded repair loops [3]. Network verification tools such as Batfish and header-space analyses provide deterministic reachability and ordering checks and are natural verifier choices in LLM-augmented NetOps prototypes; prior analyses document Batfish’s design and motivate its integration in automated pipelines [4]. Work examining what LLMs need to synthesize correct router configurations highlights sequencing and vendor-syntax difficulties that verifiers can detect and flag for repair [5].

Our study differs by executing a preregistered paired test that (a) uses shared_initial_candidate semantics exactly as registered, (b) archives raw model and validator traces for auditability, and (c) reports exact paired contingency counts together with the preregistered exact McNemar test and a paired bootstrap CI to aid interpretation in a small-discordant sample.

III. METHODOLOGY

Preregistration and estimand: The study preregistration is cnsms2026-001-gvr-batfish-prereg-v1 (manifest SHA-256: 46a223492f760950b3d06f475129ce21e52e296c4c2a1e09df3ccd50bec9f891). Primary estimand: paired_success_rate_difference_guarded_minus_baseline under shared_initial_candidate semantics. Under those semantics the identical single sampled initial model candidate is used to compute both outcomes: baseline single-shot candidate → deterministic validator pass/fail, and guarded outcome → same candidate; if it fails, the guarded arm may invoke at most one automated LLM repair call and the final

validator pass/fail is recorded. This isolates the effect of the allowed validator-triggered repair step for a fixed initial candidate.

Task bank and sampling: The preregistered task bank deterministically produced 300 intent→CLI pairs (150 Cisco-like; 150 Juniper-like) using a seeded synthetic task generator combined with public NetConfEval-style examples; per-task generator ids, seeds, and manifest entries are archived. Inclusion/exclusion, retry, and missingness rules followed the preregistration: provider/API transient failures allowed up to two automatic retries; pairs where both arms failed due to provider/infrastructure errors were excluded from the primary analysis. The execution manifest records `planned_episode_count = 600` and `completed_episode_count = 532`; missingness and reconciliation artifacts are archived.

Model, orchestration, and sampling disclosure: The declared model was `gpt-5-mini` (execution/model_configuration.json field `declared_model_version = "gpt-5-mini"`). Archived configuration fields include `max_output_tokens = 1024`, `maximum_attempts_per_call = 1`, `provider = openai_responses`, and `temperature = null` (unset). Provider accounting (deterministic_reconciliation.json) records `hosted_model_call_event_count = 306`, `completed_hosted_model_call_event_count = 272`, and `failed_hosted_model_call_event_count = 34`. Because temperature was unset and no centralized deterministic sampling seed was archived for provider calls, nondeterministic sampling cannot be excluded for recorded model calls; we note this explicitly in the Disclosure and Limitations.

Deterministic validator semantics and scoring rules: The binary oracle used across episodes was deterministic_netops_validator_v5. For each candidate the validator performed: (1) CLI syntactic parsing/normalization; (2) per-task required-sequence checks (expected_sequence vs parsed commands); (3) constrained-field touch checks (allowed_fields, allowed_touched_fields); and (4) reachability/constraint validation on the synthetic topology model. Each episode archives validation_before and validation_after traces containing `parsed_command_count`, `required_command_count`, `observed_touched_field_count`, violation codes, and normalized_configuration; the validator’s pass/fail output is the primary binary endpoint. The confirmatory scoring rule was ‘full_intent_constraint_validation’ as recorded in analysis artifacts.

Repair loop mechanics (bounded, archived): Per the preregistration the repair stage is bounded to at most one automated LLM repair call per task when the initial candidate fails the validator. The repair prompt includes validator failure codes and task constraints and requests a corrected configuration; repair provider traces and repair_prompt_sha256 are archived when invoked. Stage accounting shows `shared_initial_generation = 300` and `repair = 6` (six repair calls executed). Repair artifacts and validator traces for repaired episodes are included in execution/scoring and representative artifacts described below.

Analysis plan and inferential methods: The prespecified primary test is the exact McNemar two-sided test on paired binary outcomes at $\alpha=0.05$. Interval estimation was preregistered as a paired bootstrap percentile CI ($B=10,000$, $\text{seed}=1729$) for the paired difference to quantify magnitude and uncertainty. Missingness handling followed the preregistered ‘complete_pair_primary’ rule: both-arm provider failures were excluded from the primary test; archived sensitivity analyses treat excluded episodes conservatively as failures. Secondary estimands included `median_repair_iterations_per_task` and `median end-to-end latency`; per-episode latency was not archived and thus could not be evaluated. We included the paired bootstrap CI alongside McNemar because exact McNemar p-values can be discrete and conservative with few discordant pairs, whereas a bootstrap CI summarizes empirical paired-difference variability (see Results for the observed small-discordant behavior).

IV. RESULTS

Execution accounting and sample flow: The preregistered plan targeted 300 independent intent→CLI tasks (600 episodes across two conditions). The run recorded `planned_episode_count = 600` and `completed_episode_count = 532`; provider/infrastructure transient failures produced `failed_episode_count = 68`. Applying the preregistered ‘complete_pair_primary’ exclusion rules (exclude both-arm provider/infrastructure failures) yielded complete paired units analysed = 266. The analysis pipeline’s provider accounting indicates `hosted_model_call_event_count = 306` total model-call events across the run, with `completed_hosted_model_call_event_count = 272` and `failed_hosted_model_call_event_count = 34`; stage accounting records `shared_initial_generation = 300` and `repair = 6`.

Primary paired outcomes and contingency counts: For the 266 complete pairs the validator outcomes produced the following contingency counts: $n_{11} = 260$ (both baseline and guarded pass), $n_{10} = 5$ (guarded pass only), $n_{01} = 0$ (baseline pass only), and $n_{00} = 1$ (both fail). These counts yield observed per-condition proportions `baseline = 260/266 = 0.9774436090` and `guarded = 265/266 = 0.9962406015`; the paired (guarded - baseline) difference = 0.01879699248. The exact McNemar two-sided test applied to the discordant pair counts ($n_{10} = 5$, $n_{01} = 0$) produced $p = 0.0625$. Because the exact two-sided McNemar p for observed discordants is 0.0625 (1/16), it narrowly misses the preregistered $\alpha = 0.05$ rejection threshold.

Interpreting the small-discordant pattern and the two inferential summaries: The discordant total ($n_{10} + n_{01} = 5$) is small. The exact two-sided McNemar test calculates two-sided probability mass under a $\text{Binomial}(\text{total}=5, p=0.5)$ for outcomes as or more extreme than the observed ($k \geq 5$ or $k \leq 0$), yielding $p = 2 * (0.5^5) = 0.0625$. In contrast, the paired bootstrap percentile CI ($B = 10,000$ resamples; $\text{seed} = 1729$) for the paired difference is [0.0037593985, 0.0375939850] and excludes zero. Both summaries are valid descriptions of uncertainty; the discrete exact test is conservative when

discordant counts are few, while the bootstrap CI summarizes empirical resampling variability. We therefore report both: the preregistered hypothesis test (McNemar) did not reject at $\alpha = 0.05$, while the paired bootstrap CI suggests a small positive effect whose lower bound is barely above zero for the observed sample.

Repair invocation, yield, and derived operational quantities: Repair was invoked in 6 episodes (stage_counts: repair = 6). The contingency pattern (n10 = 5) indicates that at most five of those repair invocations contributed to additional guarded successes among the 266 analysed pairs; the run-level accounting (6 repair calls vs 5 discordant guarded wins) is consistent with a small number of successful repairs and at most one repair that did not produce a guarded pass among the complete pairs. Median repair iterations per task across complete pairs is 0 (most tasks did not require repair). A useful derived metric is the number needed to treat (NNT) interpretation: $1 / \text{absolute_risk_reduction} = 1 / 0.01879699248 \approx 53.2$, i.e., roughly 53 guarded tasks (under this run’s conditions and shared_initial_candidate semantics) were associated with one additional verifier-approved configuration relative to baseline. This NNT is a descriptive operational scalar and should be interpreted in context: with a very high baseline pass rate, the marginal correction yield per repair opportunity is small.

Representative repair case diagnostics: Representative artifacts illustrate typical failure and repair behavior. Task-000008 (difficulty: "extreme", required_command_count = 9) provides a concrete example: the shared initial candidate contained a truncated final token line ("interface up-link2\n") and failed the validator’s parsed_command_count vs required_command_count check; the automated repair call produced a corrected candidate that completed the previously truncated command and included the required edge1 sequence. The guarded final validator trace for task-000008 shows validation_after.valid = true and violation_count = 0. These per-episode validator traces record parsed_command_count, required_command_count, observed_touched_field_count, normalized_configuration, and violation codes, enabling targeted audit of repair logic and verifier coverage for each repaired episode.

Sensitivity to missingness and preregistered conservative handling: Thirty-four both-arm episodes were excluded from the primary confirmatory analysis (complete_pairs = 266) per preregistered rules. A conservative sensitivity analysis that treats those 34 excluded both-arm episodes as failures (complete N = 300) yields baseline = $260/300 = 0.866667$ and guarded = $265/300 = 0.883333$; the discordant counts among the complete pairs remain n10 = 5, n01 = 0, producing the same exact McNemar $p = 0.0625$ for the paired test on the complete-pair subset. This shows the primary inferential conclusion (nonrejection under exact McNemar) is robust to preregistered conservative missingness treatment, while the absolute per-condition proportions change due to inclusion of excluded episodes as failures.

Quantitative artifact summaries and reproducibility metadata used in Results: All numerical summaries re-

ported above derive directly from archived analysis artifacts: the paired contingency counts, condition summaries, missingness summary, and provider-call accounting produced the reported totals (complete_pairs = 266; baseline_success_count = 260; guarded_success_count = 265; hosted_model_call_event_count = 306; repair_calls = 6; both_missing_pairs = 34). The paired bootstrap used $B = 10,000$ resamples with seed = 1729 (bootstrap parameters archived) to produce the reported percentile CI. Per-episode validator traces and scoring JSONs include normalized_configuration, parsed_command_count, required_command_count, and violation codes for each episode and can be used to reproduce failure-mode breakdowns or to recalibrate validator rule coverage.

Limitations specific to the observed Results: (1) High baseline pass rate produced few discordant pairs: rare failures reduce the statistical power to detect small absolute improvements under the preregistered exact McNemar test. (2) Repair calls were infrequent (6/300) in this corpus; larger absolute improvements from GVR are more plausible on corpora with lower first-pass accuracy. (3) The NNT and repair yield reported are conditional on shared_initial_candidate semantics (the same initial sample across arms) and the particular model sampling/settings used; different sampling regimes or constrained first-pass prompts could change the yield. (4) Per-episode latency was not archived and thus the preregistered latency estimand could not be evaluated; timing instrumentation is recommended for future runs. (5) Validator coverage is finite: violations outside deterministic_netops_validator_v5’s rule battery are unmeasured here.

V. DISCUSSION

The experiment produced a small absolute improvement in verifier pass-rate when the allowed repair stage could operate on a shared initial candidate: guarded - baseline = 0.01879699248 (n11=260, n10=5, n01=0, n00=1). Practically, this pattern—few discordant pairs with all discordance in the guarded→pass direction—indicates that (a) the initial LLM candidate was correct for most tasks in this corpus, and (b) the bounded repair stage occasionally corrected defects that the initial candidate incurred. The artifact accounting (repair calls = 6; hosted_model_call_event_count = 306) confirms repairs were rare relative to total generation activity, explaining the modest absolute gain.

From a statistical perspective the run highlights an important small-sample inference nuance that directly affects operational interpretation. Exact McNemar (two-sided) returned $p = 0.0625$ under the preregistered $\alpha=0.05$ decision rule, while a paired bootstrap percentile CI ($B=10,000$, seed=1729) produced a 95% interval that narrowly excludes zero. These outcomes are consistent: with very few discordant pairs (here 5 vs 0), the exact two-sided McNemar test is discrete and can be conservative; bootstrap resampling of paired differences summarizes empirical variability and can produce a CI excluding zero even when the exact discrete p slightly exceeds 0.05. For readers and practitioners this means that reporting both formal

hypothesis outcomes and interval estimates (as archived here) gives a clearer picture: the effect magnitude is small but the CI suggests the effect is positive with narrow uncertainty in the observed corpus, whereas the exact hypothesis test under a strict α threshold does not claim formal rejection.

Operationally, artifact evidence supports three practical inferences grounded in this run. First, when first-pass generator accuracy is high ($\approx 97.7\%$ baseline here), bounded automated repair offers limited marginal benefit because few failures remain to correct; this is exactly what we observed (6 repair calls on 300 tasks). Second, the marginal utility of GVR will be larger when (i) first-pass accuracy is lower, (ii) verifier coverage detects failure modes common in the target workload, or (iii) the repair mechanism is allowed more than one iteration—conditions not present in this bounded run. Third, the design of the verifier strongly conditions measured benefit: `deterministic_netops_validator_v5` enforces a finite, archived battery of syntactic/sequence/constraint/reachability checks; any semantic failure mode outside that battery is neither detected nor credited to the GVR pipeline. In short, GVR’s measured improvement depends both on how often the generator errs and on whether the verifier can detect the errors that matter operationally.

Threats to inference and external validity remain artifact-grounded. The preregistered `shared_initial_candidate` estimand isolates the effect of validator-triggered repair for a fixed initial sample; it does not measure improvements obtainable by engineering the first pass (e.g., constrained prompting, constrained decoding, or multiple independent draws). Our execution manifest and reconciliation artifacts explicitly record `stage_counts` (`shared_initial_generation` = 300, `repair` = 6) and the lack of independent first-pass generations for separate arms, so extrapolating to comparisons that change initial sampling is unsupported by these artifacts. Missingness also reduced effective sample size: 34 both-arm provider failures were excluded per preregistered rules, lowering precision and power; `deterministic_reconciliation.json` and `analysis/missingness_summary.csv` archive these counts. Finally, per-episode latency was not recorded in per-task artifacts and therefore the preregistered latency secondary estimand could not be evaluated—this absence constrains operational claims about responsiveness or real-time applicability.

Design and research recommendations informed by these artifacts. (1) For stronger causal evidence about prompting or sampling strategies, future preregistrations should include additional independent arms that draw separate initial candidates (`independent_generation` semantics) or include constrained-first-pass prompting; these require separate initial model calls per arm and differ from the `shared_initial_candidate` estimand used here. (2) To assess operational trade-offs, pipeline runs must archive per-episode timing (`generation`→`verification`→`repair`→`final validation`) so latency and throughput can be measured; our provider accounting shows model call counts and failures but not per-episode

latencies. (3) Because verifier coverage materially shapes measured benefit, future studies should expand the verifier rule set and publish clear catalogs of detectable failure codes; the per-episode validator traces archived in `execution/scoring/*.json` enable such audits and should be exploited to extend verifier coverage and to quantify verifier false negatives. (4) When baseline pass rates are high, consider targeting task corpora with intentionally lower baseline accuracy (e.g., out-of-distribution intents, noisier prompts, or reduced grounding) to increase discordant counts and thereby statistical power to detect repair benefits.

Concluding synthesis: the archived artifacts show that a bounded GVR pipeline can correct occasional deterministic-validator failures for a fixed initial candidate, but when the generator’s first-pass accuracy is already high and the verifier’s coverage is finite, the absolute operational improvement is small. The run’s reconciled accounting, validator traces, repair provider traces, and contamination checks (no exact public duplicates flagged) provide a reproducible empirical substrate for these conclusions and for follow-on experiments that expand sampling semantics, verifier coverage, or repair budgets.

VI. LIMITATIONS

- `Shared_initial_candidate` semantics: both arms used the same single sampled initial candidate per preregistration; the estimand measures validator-triggered repair benefit for that fixed initial sample and does not evaluate independent first-pass generations or constrained first-pass prompting strategies.
- Missingness and sample reduction: planned $N=300$ pairs; after infrastructure/provider exclusions 266 pairs were complete and 34 both-arm episodes were excluded per preregistered rules, reducing effective sample size and precision.
- Small discordant counts: primary inference used exact McNemar with few discordants ($n_{10}=5$, $n_{01}=0$); conclusions are sensitive to inferential choice and limited power.
- Validator coverage: `deterministic_netops_validator_v5` enforces a finite set of syntactic/semantic/reachability rules; semantic violations outside its coverage are possible and unmeasured.
- Runtime nondeterminism and sampling: archived `execution/model_configuration.json` records `temperature` = null and no deterministic seed; nondeterministic sampling cannot be excluded for recorded model calls.
- H2 latency not evaluated: per-episode end-to-end latency was not present in archived per-task artifacts and therefore the preregistered latency bound (median ≤ 60 s) could not be assessed.
- External validity: task corpus derives from public NetConfEval-style examples and a seeded synthetic generator limited to two vendor-dialect clusters; results do not imply universal benefit across other vendors, larger topologies, or production deployments.

VII. CONCLUSION

We executed a preregistered paired experiment that evaluated a GVR pipeline (gpt-5-mini + deterministic_netops_validator_v5 + ≤ 1 automated repair) on intent→CLI tasks under shared_initial_candidate semantics. Across 266 analysed pairs the guarded pipeline improved validator pass rate by 0.01879699248 but did not meet the preregistered exact-McNemar $\alpha=0.05$ decision rule ($p=0.0625$); a paired bootstrap CI ($B=10,000$, $\text{seed}=1729$) narrowly excludes zero. Repair calls were rare (6/300) and median repair iterations per task = 0. Per-episode latency was not archived and could not be assessed. All execution and analysis artifacts (raw responses, validator traces, provider call accounting, and reconciliation manifests) are archived in the evidence bundle to support reanalysis and follow-on work.

References [1] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, and M. Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?," in *Proc. ACM*, 2024, doi: 10.1145/3656296. [2] S. Y. Long, J. Tan, B. Mao, F. Tang, Y. Li, M. Zhao, and N. Kato, "A Survey on Intelligent Network Operations and Performance Optimization Based on Large Language Models," *IEEE Commun. Surveys & Tutorials*, 2025, doi: 10.1109/comst.2025.3526606. [3] R. Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments," *SSRN*, 2026, doi: 10.2139/ssrn.6754899. [4] M. Brown, A. Fogel, D. Halperin, V. Heorhiadi, R. Mahajan, and T. Millstein, "Lessons from the evolution of the Batfish configuration analysis tool," *Proc.*, 2023, doi: 10.1145/3603269.3604866. [5] R. Mondal, A. Tang, R. Beckett, T. Millstein, and G. Varghese, "What do LLMs need to Synthesize Correct Router Configurations?," *Proc.*, 2023, doi: 10.1145/3626111.3628194.

DISCLOSURE STATEMENT

Disclosure (concise): Declared model: gpt-5-mini (execution/model_configuration.json archived; declared_model_version = "gpt-5-mini"). Orchestration adapter: hosted_netops_gvr_v1. Deterministic validator: deterministic_netops_validator_v5. Preregistration: cnsms2026-001-gvr-batfish-prereg-v1 (manifest SHA-256: 46a223492f760950b3d06f475129ce21e52e296c4c2a1e09df3ccd50bec9f891). Initial master prompt immutable reference: provenance/master_prompt.txt (SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582b b39b15ba). Human role and autonomy boundary: before the autonomy lock a human Research Director selected the topic family and approved pre-lock infrastructure development; after the autonomy lock the registered pipeline autonomously performed literature selection, preregistration, experimental execution, analysis, manuscript drafting, AI peer review simulation, and revision. Nondeterminism disclosure: archived execution/model_configuration.json records temperature = null (unset) and no deterministic sampling seed is archived;

therefore nondeterministic sampling of model outputs cannot be excluded for the recorded provider calls. Execution and analysis artifacts, logs, and hash manifests are archived in the evidence bundle referenced in the preregistration.

REFERENCES

- [1] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?," 2024, 10.1145/3656296
- [2] S.Y. Long, Jingjing Tan, Bomin Mao, Fengxiao Tang, Yangfan Li, Ming Zhao, Nei Kato, "A Survey on Intelligent Network Operations and Performance Optimization Based on Large Language Models", 2025, 10.1109/comst.2025.3526606
- [3] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.2139/ssrn.6754899
- [4] Matt Brown, Ari Fogel, Daniel Halperin, Victor Heorhiadi, Ratul Mahajan, Todd Millstein, "Lessons from the evolution of the Batfish configuration analysis tool", 2023, 10.1145/3603269.3604866
- [5] Rajdeep Mondal, Alan Tang, Ryan Beckett, Todd Millstein, George Varghese, "What do LLMs need to Synthesize Correct Router Configurations?," 2023, 10.1145/3626111.3628194